

DynamoDB

Seven Databases in Seven Weeks — Capítulo 7

Base de datos NoSQL key-value gestionada por AWS. Emulada localmente con LocalStack.

DynamoDB es una **base de datos NoSQL administrada en la nube** a través de **Amazon Web Services (AWS)**. Amazon se encarga de la infraestructura, el escalado, las copias de seguridad y la disponibilidad. **usamos LocalStack**, el cual es un **emulador local de los servicios de AWS**. Al agregar el parámetro **--endpoint-url**, obligamos a la consola a apuntar a nuestro emulador en lugar de ir a los servidores reales de Amazon

1.2. Modelo de datos: Key-Value Plus

Esta solución se define como una **arquitectura NoSQL de tipo "key-value plus"**. Aunque guarda **similitud con motores clave-valor puros (como Redis)**, **ofrece funciones avanzadas de filtrado**. Sus pilares técnicos son:

- **Tablas:** Son el **análogo a las tablas relacionales**; requieren definición previa de su esquema de claves.
- **Items:** Representan las **filas de la tabla**. Poseen una naturaleza flexible (schemaless), exceptuando los campos de indexación.
- **Atributos:** Cumplen el rol de las **columnas tradicionales** pero sin rigidez estructural, salvo por la clave primaria.
- **Claves:** Eje central del diseño que rige la partición física y la eficiencia en las lecturas.

1.3. Tipos de Claves

El motor permite implementar dos modalidades de clave primaria:

- **Hash Key (Partition Key):** Clave de identificación **única y simple**. Gestiona la distribución física del dato; **óptima para búsquedas puntuales**.
- **Composite Key (Hash + Range Key):** Clave de tipo **compuesta**. **Habilita la ejecución de queries** por rango sobre una misma partición. Útil en telemetría o registros históricos.

1.4. Modelo de Consistencia

El motor de **DynamoDB** permite elegir entre **dos esquemas de consistencia** para las lecturas, configurables individualmente por petición:

- **Eventualmente consistente (Eventually Consistent):** Prioriza la **velocidad de respuesta**; existe la posibilidad de recibir información con desfase mínimo (milisegundos). **Costo: 1 RCU** por bloque de 4KB.
- **Fuertemente consistente (Strongly Consistent):** Asegura la recuperación del **valor más actualizado**. **Costo: 2 RCU** por bloque de 4KB. Su disponibilidad puede verse comprometida ante interrupciones de red.

0. Setup — LocalStack con Docker

PowerShell (desde la carpeta DynamoDB/)

```
docker-compose up -d
```

```
# Verificar salud del servicio:
Invoke-WebRequest -Uri http://localhost:4566/_localstack/health |
Select-Object -ExpandProperty Content
```

Configurar credenciales para LocalStack (solo primera vez):

PowerShell

```
# Variables de entorno (LocalStack acepta cualquier valor):
$env:AWS_ACCESS_KEY_ID      = "test"
$env:AWS_SECRET_ACCESS_KEY  = "test"
$env:AWS_DEFAULT_REGION     = "us-east-1"

# Verificar que aws CLI funciona:
aws dynamodb list-tables --endpoint-url http://localhost:4566
```

NOTA: Todos los comandos usan `--endpoint-url http://localhost:4566` para apuntar a LocalStack en lugar de AWS real. Ejecutar todos desde la carpeta `DynamoDB/` porque los comandos referencian archivos locales con `file://`.

DÍA 1 — CRUD y Shopping Cart

El libro modela un carrito de compras y luego una tabla de libros con clave compuesta para demostrar consultas por rango.

1.1 Tabla ShoppingCart — Hash Key simple

PowerShell

```
# Verificar que no hay tablas:
aws dynamodb list-tables --endpoint-url http://localhost:4566

# Crear tabla con hash key simple (ItemName):
aws dynamodb create-table `
  --table-name ShoppingCart `
  --attribute-definitions AttributeName=ItemName,AttributeType=S `
  --key-schema AttributeName=ItemName,KeyType=HASH `
  --provisioned-throughput ReadCapacityUnits=1,WriteCapacityUnits=1 `
  --endpoint-url http://localhost:4566

# Inspeccionar la tabla:
aws dynamodb describe-table --table-name ShoppingCart --endpoint-url
http://localhost:4566
```

1.2 INSERT — put-item

NOTA: Los items se pasan desde archivos JSON (file://) para evitar problemas de quoting en PowerShell.

PowerShell

```
aws dynamodb put-item --table-name ShoppingCart --item file://item-elmo.json
--endpoint-url http://localhost:4566
aws dynamodb put-item --table-name ShoppingCart --item
file://item-buick.json --endpoint-url http://localhost:4566
aws dynamodb put-item --table-name ShoppingCart --item
file://item-burns.json --endpoint-url http://localhost:4566
```

Contenido de los archivos item-*.json:

```
item-elmo.json → {"ItemName":{"S":"Tickle Me Elmo"}}
item-buick.json → {"ItemName":{"S":"1975 Buick LeSabre"}}
item-burns.json → {"ItemName":{"S":"Ken Burns: the Complete Box Set"}}
```

1.3 SCAN, GET y DELETE

PowerShell

```
# SCAN — equivale a SELECT * (usar con cuidado en produccion):
aws dynamodb scan --table-name ShoppingCart --endpoint-url
http://localhost:4566

# GET — obtener por hash key:
aws dynamodb get-item --table-name ShoppingCart `
  --key file://key-elmo.json --endpoint-url http://localhost:4566

# GET con lectura fuertemente consistente:
```

```
aws dynamodb get-item --table-name ShoppingCart `
  --key file://key-elmo.json --consistent-read --endpoint-url
http://localhost:4566

# DELETE:
aws dynamodb delete-item --table-name ShoppingCart `
  --key file://key-elmo.json --endpoint-url http://localhost:4566

# Verificar que quedan 2 items:
aws dynamodb scan --table-name ShoppingCart --endpoint-url
http://localhost:4566
```

1.4 Tabla Books — Hash + Range Key (clave compuesta)

La clave compuesta permite consultas por rango sobre items con el mismo hash key.

PowerShell

```
aws dynamodb create-table `
  --table-name Books `
  --attribute-definitions `
    AttributeName=Title,AttributeType=S `
    AttributeName=PublishYear,AttributeType=N `
  --key-schema `
    AttributeName=Title,KeyType=HASH `
    AttributeName=PublishYear,KeyType=RANGE `
  --provisioned-throughput ReadCapacityUnits=1,WriteCapacityUnits=1 `
  --endpoint-url http://localhost:4566

# Insertar tres ediciones de Moby Dick:
aws dynamodb put-item --table-name Books --item file://item-moby-1851.json
--endpoint-url http://localhost:4566
aws dynamodb put-item --table-name Books --item file://item-moby-1971.json
--endpoint-url http://localhost:4566
aws dynamodb put-item --table-name Books --item file://item-moby-2008.json
--endpoint-url http://localhost:4566
```

1.5 QUERY — consultas por rango

NOTA: Las expresiones de valores se pasan desde archivos JSON. Archivos: *expr-books-1980.json* y *expr-books-1900.json*.

PowerShell

```
# Libros publicados despues de 1980:
aws dynamodb query --table-name Books `
  --expression-attribute-values file://expr-books-1980.json `
  --key-condition-expression "Title = :title AND PublishYear > :year" `
  --endpoint-url http://localhost:4566

# Proyeccion: solo ISBN de libros post-1900:
aws dynamodb query --table-name Books `
  --expression-attribute-values file://expr-books-1900.json `
  --key-condition-expression "Title = :title AND PublishYear > :year" `
  --projection-expression "ISBN" `
  --endpoint-url http://localhost:4566
```

Contenido de los archivos expr-books-*.json:

```
expr-books-1980.json → {"title":{"S":"Moby Dick"},"year":{"N":"1980"}}
expr-books-1900.json → {"title":{"S":"Moby Dick"},"year":{"N":"1900"}}
```

1.6 Tabla BonestormData — Local Secondary Index (LSI)

El libro modela un videojuego: Username es hash key, Date es range key. Se agrega un LSI sobre HighScore para consultar por puntaje.

Partition key		Local secondary index	No index
Username (hash key)	Date (range key)	High score	Avatar
h4x0rjimduggan	04012017	751	Lion
h4x0rjimduggan	05032017	852	Skull
beboprocksteady75	04262017	451	Raptor
beboprocksteady75	05032017	562	Lion
beboprocksteady75	06122017	567	Narwhal

Si no tuviéramos el LSI sobre highScore, la consulta debería revisar los registros ordenados por fechas lo que sería lento, en cambio con el LSI sobre highScore la consulta de los días donde se superan los 800 puntos se hace más eficiente

PowerShell

```
aws dynamodb create-table `
  --table-name BonestormData `
  --attribute-definitions `
    AttributeName=Username,AttributeType=S `
    AttributeName=Date,AttributeType=S `
    AttributeName=HighScore,AttributeType=N `
  --key-schema `
    AttributeName=Username,KeyType=HASH `
    AttributeName=Date,KeyType=RANGE `
  --local-secondary-indexes file://lsi-bonestorm.json `
  --provisioned-throughput ReadCapacityUnits=1,WriteCapacityUnits=1 `
  --endpoint-url http://localhost:4566

# Insertar datos:
aws dynamodb put-item --table-name BonestormData --item
file://item-h4x0r-2017-04.json --endpoint-url http://localhost:4566
aws dynamodb put-item --table-name BonestormData --item
file://item-h4x0r-2017-05.json --endpoint-url http://localhost:4566
aws dynamodb put-item --table-name BonestormData --item
file://item-pity.json --endpoint-url http://localhost:4566

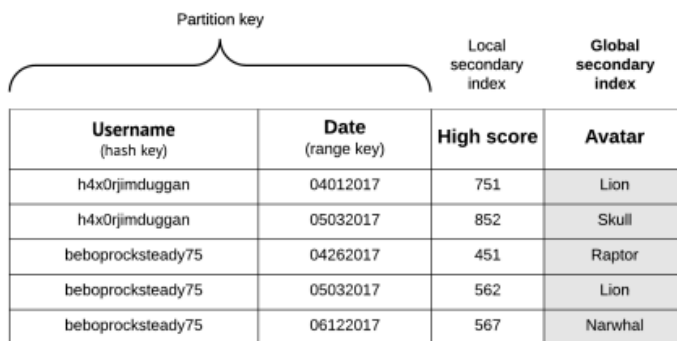
# Consulta LSI: dias donde h4x0rjimduggan supero 800 puntos:
aws dynamodb query --table-name BonestormData --index-name HighScoreIndex `
  --expression-attribute-values file://expr-bonestorm-score.json `
  --key-condition-expression "Username = :user AND HighScore > :score" `
```

```
--projection-expression "#D" `
--expression-attribute-names file://names-date.json `
--endpoint-url http://localhost:4566
```

NOTA: `#D` es un alias para `'Date'` porque es palabra reservada en DynamoDB. `names-date.json` contiene: `{"#D": "Date"}`

1.7 Global Secondary Index (GSI)

Si queremos **buscar** a todos los **usuarios que juegan con un avatar específico** (por ejemplo, el Avatar de León), **no** podemos hacerlo de **forma eficiente** porque **Avatar** es solo un **atributo común**, **no** una **clave principal**. Deberíamos buscarlo con un Scan lo cual no es eficiente. Entonces se **construye un índice GSI sobre el atributo Avatar** para **maximizar la eficiencia** en la **búsqueda** de los **usuarios que juegan con un avatar específico**



The diagram illustrates the structure of a DynamoDB table and its indexes. A bracket labeled 'Partition key' spans the 'Username' and 'Date' columns. A bracket labeled 'Local secondary index' spans the 'High score' column. A bracket labeled 'Global secondary index' spans the 'Avatar' column.

Username (hash key)	Date (range key)	High score	Avatar
h4x0rjimduggan	04012017	751	Lion
h4x0rjimduggan	05032017	852	Skull
beboprocksteady75	04262017	451	Raptor
beboprocksteady75	05032017	562	Lion
beboprocksteady75	06122017	567	Narwhal

PowerShell

```
# Agregar GSI sobre Avatar (permite buscar por avatar sin conocer el
username):
aws dynamodb update-table --table-name BonestormData `
  --attribute-definitions AttributeName=Avatar,AttributeType=S `
  --global-secondary-index-updates file://gsi-avatar-update.json `
  --endpoint-url http://localhost:4566

# Esperar a que el GSI quede ACTIVE (puede tardar unos segundos en
LocalStack):
aws dynamodb describe-table --table-name BonestormData --endpoint-url
http://localhost:4566

# Consulta GSI: todos los que usan Avatar=Lion:
aws dynamodb query --table-name BonestormData --index-name AvatarIndex `
  --expression-attribute-values file://expr-avatar.json `
  --key-condition-expression "Avatar = :avatar" `
  --projection-expression "Username, #D" `
  --expression-attribute-names file://names-date.json `
  --endpoint-url http://localhost:4566
```

DÍA 2 — Pipeline de Streaming: Kinesis → Lambda → DynamoDB

El libro **construye un pipeline de datos de sensores IoT**:

AWS Kinesis actúa como el **servicio de streaming que recibe** grandes volúmenes de **datos en tiempo real** (como lecturas de sensores IoT). Luego, una función "Serverless" (**AWS Lambda**) actúa como **punto de puente: se dispara automáticamente para procesar esos datos** en vuelo y **finalmente guardarlos** de forma permanente en **DynamoDB**.

2.1 Crear tabla SensorData (desde JSON)

PowerShell

```
aws dynamodb create-table --cli-input-json file://sensor-data-table.json
--endpoint-url http://localhost:4566

# Verificar:
aws dynamodb describe-table --table-name SensorData --endpoint-url
http://localhost:4566
```

2.2 Crear stream Kinesis

--shard-count 1 genera una autopista para los datos de un solo carril

PowerShell

```
$env:STREAM_NAME = "temperature-sensor-data"
aws kinesis create-stream --stream-name $env:STREAM_NAME --shard-count 1
--endpoint-url http://localhost:4566

# Verificar estado (esperar ACTIVE):
aws kinesis describe-stream --stream-name $env:STREAM_NAME --endpoint-url
http://localhost:4566

# Guardar el ARN:
$env:KINESIS_STREAM_ARN = (aws kinesis describe-stream `
  --stream-name $env:STREAM_NAME `
  --endpoint-url http://localhost:4566 `
  --query "StreamDescription.StreamARN" --output text)
Write-Host "Stream ARN: $env:KINESIS_STREAM_ARN"
```

2.3 Crear rol IAM, empaquetar y subir Lambda

El rol IAM permite a AWS lambda a actuar como puente entre los servicios, lee los datos de Kinesis y los escribe en DynamoDB

PowerShell

```
$env:IAM_ROLE_NAME = "kinesis-lambda-dynamodb"

# Crear el rol IAM:
```

```

aws iam create-role `
  --role-name $env:IAM_ROLE_NAME `
  --assume-role-policy-document file://lambda-kinesis-role.json `
  --endpoint-url http://localhost:4566

# Obtener el ARN del rol:
$env:ROLE_ARN = (aws iam get-role `
  --role-name $env:IAM_ROLE_NAME `
  --endpoint-url http://localhost:4566 `
  --query "Role.Arn" --output text)

# Empaquetar la Lambda en ZIP:
Compress-Archive -Path ProcessKinesisRecords.js -DestinationPath
ProcessKinesisRecords.zip -Force

# Subir la Lambda:
aws lambda create-function `
  --region us-east-1 `
  --function-name ProcessKinesisRecords `
  --zip-file fileb://ProcessKinesisRecords.zip `
  --role $env:ROLE_ARN `
  --handler ProcessKinesisRecords.kinesisHandler `
  --runtime nodejs18.x `
  --endpoint-url http://localhost:4566

```

2.4 Invocar Lambda manualmente y conectar con Kinesis

PowerShell

```

# Probar la Lambda con el archivo de prueba incluido:
aws lambda invoke `
  --invocation-type RequestResponse `
  --function-name ProcessKinesisRecords `
  --payload fileb://test-lambda-input.txt `
  lambda-output.txt `
  --endpoint-url http://localhost:4566

# Ver resultado:
Get-Content lambda-output.txt

# Verificar que el item fue guardado en SensorData:
aws dynamodb scan --table-name SensorData --endpoint-url
http://localhost:4566

# Conectar Kinesis -> Lambda (event source mapping):
aws lambda create-event-source-mapping `
  --function-name ProcessKinesisRecords `
  --event-source-arn $env:KINESIS_STREAM_ARN `
  --starting-position LATEST `
  --endpoint-url http://localhost:4566

```

2.5 Enviar datos reales al pipeline

PowerShell

```

# Codificar JSON en Base64:

```

```
$json =  
'{"sensor_id":"sensor-1","temperature":99.9,"current_time":123456789}'  
$bytes = [System.Text.Encoding]::UTF8.GetBytes($json)  
$env:DATA = [Convert]::ToBase64String($bytes)  
  
# Enviar al stream Kinesis:  
aws kinesis put-record `  
  --stream-name $env:STREAM_NAME `  
  --partition-key sensor-data `  
  --data $env:DATA `  
  --endpoint-url http://localhost:4566  
  
# Esperar y verificar en DynamoDB:  
Start-Sleep -Seconds 5  
aws dynamodb scan --table-name SensorData --endpoint-url  
http://localhost:4566
```

DÍA 3 — IoT y Operaciones Avanzadas

El Día 3 del libro extiende el pipeline con un sistema IoT completo. Los conceptos clave:

3.1 DynamoDB Streams

Funcionalidad nativa que **captura cambios en tiempo real (inserciones, modificaciones, eliminaciones)**. Diferente de Kinesis: los Streams se activan automáticamente con cada cambio en la tabla.

PowerShell

```
aws dynamodb update-table `
  --table-name ShoppingCart `
  --stream-specification
StreamEnabled=true,StreamViewType=NEW_AND_OLD_IMAGES `
  --endpoint-url http://localhost:4566

# Ver el ARN del stream:
aws dynamodb describe-table `
  --table-name ShoppingCart --endpoint-url http://localhost:4566 --query
"Table.LatestStreamArn"
```

3.2 Update condicional (operaciones atómicas)

PowerShell

```
# Agregar item con precio:
aws dynamodb put-item --table-name ShoppingCart --item
file://item-buick-precio.json --endpoint-url http://localhost:4566

# Actualizar precio SOLO SI el item existe (condition expression):
aws dynamodb update-item --table-name ShoppingCart `
  --key file://key-buick.json `
  --update-expression "SET Precio = :newPrice" `
  --condition-expression "attribute_exists(ItemName)" `
  --expression-attribute-values file://expr-price-750.json `
  --endpoint-url http://localhost:4566

# Verificar el cambio:
aws dynamodb get-item --table-name ShoppingCart `
  --key file://key-buick.json --endpoint-url http://localhost:4566

# Intentar actualizar item INEXISTENTE (debe dar
ConditionalCheckFailedException):
aws dynamodb update-item --table-name ShoppingCart `
  --key file://key-inexistente.json `
  --update-expression "SET Precio = :newPrice" `
  --condition-expression "attribute_exists(ItemName)" `
  --expression-attribute-values file://expr-price-100.json `
  --endpoint-url http://localhost:4566
```

NOTA: *expr-price-750.json contiene {"newPrice":{"N":"750"}}. key-buick.json contiene {"ItemName":{"S":"1975 Buick LeSabre"}}.*

Resumen de Comandos

Comando aws dynamodb	Descripción
list-tables	Listar todas las tablas existentes
create-table	Crear tabla (con schema de claves y throughput)
describe-table --table-name X	Ver configuración y estado de una tabla
put-item --table-name X --item file://X.json	Insertar o reemplazar un ítem
get-item --table-name X --key file://X.json	Obtener un ítem por clave primaria
delete-item --table-name X --key file://X.json	Eliminar un ítem por clave primaria
scan --table-name X	Leer TODOS los ítems (costoso en producción)
query --table-name X ...	Consultar por clave primaria o índice
update-item --update-expression ...	Actualizar atributos con condición opcional
update-table --global-secondary-index-updates file://X.json	Agregar GSI
update-table --stream-specification ...	Habilitar DynamoDB Streams

Archivos JSON incluidos

Archivo	Contenido / Uso
item-elmo.json	Ítem: Tickle Me Elmo (ShoppingCart)
item-buick.json	Ítem: 1975 Buick LeSabre (ShoppingCart)
item-burns.json	Ítem: Ken Burns Box Set (ShoppingCart)
item-moby-1851/1971/2008.json	Tres ediciones de Moby Dick (Books)
item-h4x0r-2017-04/05.json	Partidas de h4x0rjimduggan (BonestormData)
item-pity.json	Partida de pitythefool (BonestormData)
item-buick-precio.json	Buick con atributo Precio=500 (ShoppingCart)
key-elmo.json / key-buick.json	Claves para get-item / delete-item
key-inexistente.json	Clave para demo de ConditionalCheckFailedException
expr-books-1980.json / 1900.json	Valores para query de Books por año
expr-bonestorm-score.json	Valores para query LSI de BonestormData
expr-avatar.json	Valores para query GSI de AvatarIndex
expr-price-750.json / 100.json	Valores para update-item condicional
lsi-bonestorm.json	Definición del Local Secondary Index HighScoreIndex
gsi-avatar-update.json	Payload para agregar el GSI AvatarIndex
names-date.json	Alias #D → Date (palabra reservada de DynamoDB)

sensor-data-table.json	Definición completa de la tabla SensorData
lambda-kinesis-role.json	Trust policy IAM para Lambda + Kinesis

Conceptos Clave

Término	Definición
Item	Equivalente a una fila; solo las claves son obligatorias, el resto es schemaless
Hash Key (Partition Key)	Clave que determina la partición física; distribuye los datos
Range Key (Sort Key)	Segunda clave de la clave compuesta; permite consultas por rango
LSI (Local Secondary Index)	Índice sobre atributos del mismo hash key; se define al crear la tabla
GSI (Global Secondary Index)	Índice sobre cualquier atributo; se puede agregar después de crear la tabla
RCU / WCU	Read/Write Capacity Units — 1 RCU = lectura consistente de 4KB
Scan	Lee TODOS los items; caro en producción — evitar en tablas grandes
Query	Lee items por clave primaria o índice; siempre más eficiente que Scan
Condition Expression	Condición para escrituras atómicas (attribute_exists, <, >, etc.)
Projection Expression	Qué atributos devolver (equivale a SELECT col1, col2)
DynamoDB Streams	Stream de cambios (inserts/updates/deletes) para disparar Lambda
Kinesis	Servicio de streaming; en el libro recibe datos de sensores IoT
LocalStack	Emulador local de servicios AWS — misma API sin costo ni cuenta real
file://	Prefijo AWS CLI para leer parámetros JSON desde archivo — evita problemas de quoting en PowerShell